DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 04

Lab Outcome: Develop responsive front-end applications using React components and state management.

Aim: Implement state management using Hooks and Context API; add form handling and validation.

Software used: JavaScript, NodeJS, React.

Theory: -

1. State & State Management

- **State:** Data that changes while the application runs (e.g., counters, themes, login status).
- **State Management:** The process of storing, updating, and sharing this changing data across components.

2. React Hooks & useState()

Hooks let functional components use React features:

- **useState():** Stores and updates state (const [count, setCount] = useState(0)). Calling the updater function triggers a component re-render.
- *Note:* Never mutate state directly; always use the updater function for React to detect changes.
- Other hooks: useEffect(), useContext(), useRef(), useReducer().

3. Data Flow & Prop Drilling

- **One-Way Data Flow:** Data flows downward from parent to child via **props**.
- **Prop Drilling:** The inefficient process of passing props through multiple intermediate components that don't need the data just to reach a deeply nested component.

4. The Context API

The Context API eliminates prop drilling by acting as a shared global store:

1. **createContext():** Creates the shared context space.
2. **Provider:** Wraps components to supply data (<Context.Provider value="{...}">).
3. **useContext():** Consumes the shared data directly in any nested component.

- Combining useState() and Context allows components to both read and update shared global state.

5. Local vs. Global State

- **Local State:** Restricted to a single component (e.g., form inputs via useState).
- **Global State:** Shared across multiple components at different tree levels (e.g., user profiles, application themes).

6. Modular Design & Separation of Concerns

Splitting applications into single-purpose components improves **maintainability**, **reusability**, and **predictable execution flow**.

7. Form Handling & Client-Side Validation

- **Controlled Components:** Form inputs bound directly to React state (value and onChange), making state the single source of truth.
- **Validation:** Uses e.preventDefault() to block page reloads, applies regular expressions (Regex) to check input validity, and uses short-circuit expressions for conditional error rendering.

Conclusion: -
Thus, we have designed and implemented state management using Hooks and Context API; added form handling and validation.